Intent-Consistency Action Gating for Tool-Using LLM Agents: A Lightweight Middleware Defense Against Prompt Injection

Anonymous Author(s)
Anonymous Institution
City, Country
anonymous@anonymous.org

***Abstract*—Tool-using agents built on large language models (LLMs) can be redirected by instructions hidden in tool output or tool descriptions, a risk that grows as agents reach email, calendars, and databases. Existing defenses modify the model, add a judge call per action, or require hand-authored per-tool policies. This paper presents IntentGate, a lightweight middleware gate that derives an intent contract once from the user's trusted request, before any attacker content is loaded. Every proposed tool call is scored by embedding similarity to the contract plus a deterministic rule veto, then allowed, blocked, or escalated, with no LLM call per action. The design targets the single chokepoint every attack crosses, and its policy comes from the user, not from text an attacker can touch. On InjecAgent and MCPTox, attack success falls from 19.0% to 0.7% and attack-influenced calls from 20.7% to 10.7%, at 16–20 ms p95 CPU latency and zero setup cost against 145 hand-authored baseline contracts. Pilots on a second backbone, benign-task utility, and prompt baselines are reported as preliminary.**

***Index Terms*—LLM agents, prompt injection defense, tool poisoning, action gate, intent contract, middleware, MCP**

I. Introduction

Tool-using LLM agents act through mail, calendars, databases, and code in a ReAct-style loop [1]. That reach is also the attack surface. Instructions hidden in retrieved documents or tool output can redirect the agent through indirect prompt injection [2], [3], and a malicious tool description can redirect it through tool poisoning, documented on real servers of the Model Context Protocol (MCP) [4], [5]. In both cases the agent performs actions the user never authorized, the excessive-agency risk in the OWASP Top 10 for LLM applications [6]. Current defenses sit at the model level, in a judge call per action, or in hand-authored per-tool policies, so a lightweight, model-agnostic check that derives its policy from the user's own request is missing. This paper introduces IntentGate: a middleware gate that scores every proposed tool call against an intent contract parsed once from the trusted user request, before any attacker content is loaded. The evaluation covers an injection benchmark (InjecAgent) and a tool-poisoning benchmark (MCPTox).

Prior defenses fall into two groups. Model-level methods change the model or its training data: CaMeL [7] wraps the model in a planner-executor design with capability tracking, StruQ [8] and SecAlign [9] fine-tune the model to separate instructions from data, and instruction-hierarchy training [10] teaches models to prioritize trusted instructions. These approaches need model cooperation and report no head-to-head evaluation on InjecAgent or MCPTox. Data-level methods mark or filter untrusted text: Spotlighting [11] transforms it, and guardrail toolkits [12] enforce programmable rails, but the model still makes the final call. Boundary-level methods check the tool call itself. ToolGate [13] enforces hand-authored Hoare-style contracts at the call boundary, the same chokepoint this paper uses, and its weakness is the policy source: one contract per tool (145 on InjecAgent) and none for attacker-registered tools. Progent [14] enforces symbolic privilege rules per call, but the policies are authored per tool. ClawGuard [15] derives task-specific constraints from the user's stated objective and enforces them deterministically, with a user confirmation step; it reports results on six injection benchmarks that include InjecAgent and MCPTox. Task Shield [16] adds a judge model to every action.

Two problems remain unsolved. Policy setup stays manual or indirect, and no boundary defense reports a frozen head-to-head comparison on both classes of benchmark. IntentGate derives the contract automatically from the user request, scores each call by embedding similarity plus a deterministic veto, and decides allow, block, or escalate at the call boundary, with no LLM call per action. On the frozen evaluation, attack success on InjecAgent falls from 19.0% to 0.7% and attack-influenced MCPTox calls from 20.7% to 10.7%, at 16–20 ms p95 CPU latency and zero setup cost; a ToolGate reimplementation with 145 contracts stays statistically indistinguishable from the unprotected agent.

II. Background and Related Work

A. Threat model

The threat model is a tool-using LLM agent in a ReAct-style loop [1]: given a trusted user request, the agent proposes tool calls, executes them, observes the results, and repeats until it produces a final answer. Attacker content arrives through tool descriptions (tool poisoning) or through retrieved or returned content (indirect injection). The attacker's goal is to cause the agent to execute a tool call the user did not authorize: data exfiltration, message forwarding, or state mutation. The defender's goal is to block or escalate such calls while preserving legitimate task completion. The initial user request

is assumed trustworthy; everything after it (tool definitions returned by a registry, tool outputs, injected document text) is potentially attacker-controlled. Attack content ranges from manually written text to optimized adversarial sequences [17], and earlier risk studies catalogue related failure modes in emulated agent environments [18].

B. Defenses

Model-level defenses track information flow at the model or runtime level. CaMeL [7] wraps the model in a planner-executor design with capability-based taint tracking, so data from attacker-controlled sources cannot reach tool arguments; StruQ [8] fine-tunes the model to separate instructions from data with structured queries, SecAlign [9] applies preference optimization to the same end, and instruction-hierarchy training [10] teaches the model to prioritize privileged instructions. All of these require model cooperation and publish no head-to-head adversarial evaluation on InjecAgent or MCPTox. Data-level defenses mark or transform untrusted text: Spotlighting [11] converts retrieved content into clearly delimited forms, and guardrail toolkits such as NeMo Guardrails [12] enforce programmable rails around the model, but the model still makes the final call. Detection methods add training or inference cost: BIPIA fine-tunes a detector alongside its benchmark [19], and MELON re-executes with masked tools and compares outputs [20].

Judge-based defenses ask a second language model to vet each proposed call [16]. They are flexible, but they add a language-model call (with cost and latency) on every action, introduce a new attack surface (the judge can be prompt-injected), and produce implicit policies that are hard to audit. Contract-based defenses come closest to the placement of this work. ToolGate [13] enforces hand-authored Hoare-style pre/post-condition contracts per tool at the tool-call boundary, the same chokepoint used here; its weakness is the policy source, because contracts are written manually per tool, coverage is incomplete (65/801 MCPTox tools, 8.1% distinct and 37.2% availability-weighted) and attacker-registered tools have no contract at all. Progent [14] also enforces symbolic per-call privilege rules at the tool-call layer (fail-closed, no LLM in the decision loop), but its policies are authored manually or by an LLM per tool rather than derived from the user's request, and it is evaluated on AgentDojo and ASB rather than InjecAgent and MCPTox. ClawGuard [15] derives task-specific constraints from the user's stated objective and enforces them deterministically, with a user confirmation step; it reports results on six injection benchmarks including InjecAgent and MCPTox, so this paper does not claim to be the first to derive policy from the request. Prompt Flow Integrity [21] prevents privilege escalation through agent isolation and data tracking, evaluated on AgentDojo and AgentBench. TrustAgent [22] uses a hand-authored constitution and is evaluated by emulation, not live tool execution. A recent design-pattern survey [23] argues that the safest agent designs never let untrusted data steer privileged actions, which matches the boundary-placement argument of this paper.

AgentDojo [24], InjecAgent [25], and Agent Security Bench [26] provide the injection benchmarks; MCPTox [4] provides the tool-poisoning benchmark, and the MCP specification itself ships no defense against poisoned registrations [5], [27]. Zhan et al. [28] show that adaptive attackers break defenses which look strong under static attacks, a limitation this paper addresses by scoping the primary claims to static attacks and reporting adaptive-attacker work as ongoing.

C. Research gap

The missing piece is a boundary gate that needs no model change, no per-action LLM call, and no per-tool policy, and that is compared head-to-head against another boundary defense on both an injection benchmark and a tool-poisoning benchmark. Concurrent work (ClawGuard, Task Shield) derives task-scoped policies from the user's request; ClawGuard is evaluated on both benchmarks named above. The contribution of this paper is a lightweight, CPU-only gate with no LLM in the per-action loop, whose policy is derived automatically from the user's request, plus a frozen reproducible head-to-head against a ToolGate reimplementation on both benchmarks and the direct measurement of manual contract setup cost (0 vs. 145).

III. Method

A. System overview and ordering invariant

IntentGate sits between the agent and the executor. Figure 1 shows the pipeline. The ordering invariant is load-bearing: the intent contract is parsed from the trusted user request *before* tool definitions or any attacker content (injected observations, poisoned tool descriptions) are assembled, and tool outputs are never passed to the parser. Every proposed tool call, user-initiated and attacker-influenced alike, passes through `GateMiddleware.execute()` with no bypass path.

B. Intent parser and contract schema

A trusted intent parser (an LLM at temperature 0, JSON mode, one repair retry) reads the user request once per session and emits an intent contract

$$C = (A, K, H), \tag{1}$$

where A is the set of allowed (entity, operation, resource) elements, K is a set of numeric and path constraints, and H is a set of high-risk flags. A call that touches an element outside A, violates a constraint in K, or matches a flag in H fails the rule check and triggers the veto. The schema is versioned and frozen for the evaluation. The contract is embedded once per session and reused; only the per-call request is embedded at decision time, which keeps the agent prompt cacheable and the gate out of the prompt.

C. Scorer

Each proposed tool call c is scored against the contract. The semantic component is the cosine similarity between the contract embedding e_C and the call embedding e_c,

$$S_{\text{sem}} = \cos(e_C, e_c) = \frac{e_C \cdot e_c}{\|e_C\| \, \|e_c\|}, \tag{2}$$

TABLE I
RELATED-WORK COMPARISON: POLICY SOURCE, ENFORCEMENT, PER-ACTION LLM USE, BENCHMARKS, AND SETUP COST.

System	Policy source	Enforcement	LLM/action	Benchmarks	Setup cost
CaMeL	Model-level	Capability taint tracking	No	Not reported	Model change
StruQ	Model-level	Fine-tuned model	No	Not reported	Model change
Task Shield	Task-scoped (LLM-derived)	LLM-as-judge	Yes	Not reported	Per-task LLM calls
ClawGuard	Task-scoped (from user objective)	Deterministic rules	No	InjecAgent, MCPTox	Once per request
Progent	Manual/LLM-authored DSL	Deterministic symbolic	No	AgentDojo, ASB	Per-tool policies
ToolGate	Manual Hoare contracts	Deterministic symbolic	No	Not reported	145 contracts (InjecAgent)
Ours	**Auto-derived from user request**	**Embedding + rule veto**	No	InjecAgent, MCPTox	**0 contracts**

Algorithm 1 Gate decision for one proposed tool call

Require: contract $C = (A, K, H)$ with embedding e_C; tool call c with embedding e_c; parameters α, τ, δ
Ensure: decision d: allow, block, or escalate
$S_{\text{sem}} \leftarrow \cos(e_C, e_c)$
$S_{\text{rule}} \leftarrow 1$ if c satisfies A and K, else 0
$S \leftarrow \alpha S_{\text{sem}} + (1-\alpha) S_{\text{rule}}$
if $S_{\text{rule}} = 0$ or $S < \tau - \delta$ **then**
  $d \leftarrow$ block ▷ fail-closed hard veto
else if $S < \tau$ **then**
  $d \leftarrow$ escalate ▷ human-review band
else
  $d \leftarrow$ allow
end if
return d

and the rule component is deterministic, $S_{\text{rule}} \in \{0, 1\}$, with $S_{\text{rule}} = 1$ when c passes all constraints in K and $S_{\text{rule}} = 0$ when the veto fires. The combined score is

$$S = \alpha\, S_{\text{sem}} + (1 - \alpha)\, S_{\text{rule}}, \tag{3}$$

with default $\alpha = 0.7$. The threshold $\tau = 0.75$ and half-width $\delta = 0.1$ were fixed on a 50-call pilot before any benchmark run; this paper reports the operating point and does not claim Neyman–Pearson optimality or cross-split transfer.

D. Gate middleware and escalate band

The middleware decides per call:

$$\text{decision}(c) = \begin{cases} \text{BLOCK}, & S_{\text{rule}} = 0 \text{ or } S < \tau - \delta, \\ \text{ESCALATE}, & S_{\text{rule}} = 1 \text{ and } \tau - \delta \le S < \tau, \\ \text{ALLOW}, & \text{otherwise}, \end{cases} \tag{4}$$

where the block case is fail-closed and escalate routes to human review, counted as block in benchmark mode and shown to the user in demo mode. Algorithm 1 lists the full procedure. Of escalated calls, 60.7% are attacker calls, 14.8% user calls, and 23.5% unclassified. The gate adds no LLM call per action; measured p95 per-call latency is 16–20 ms on CPU.

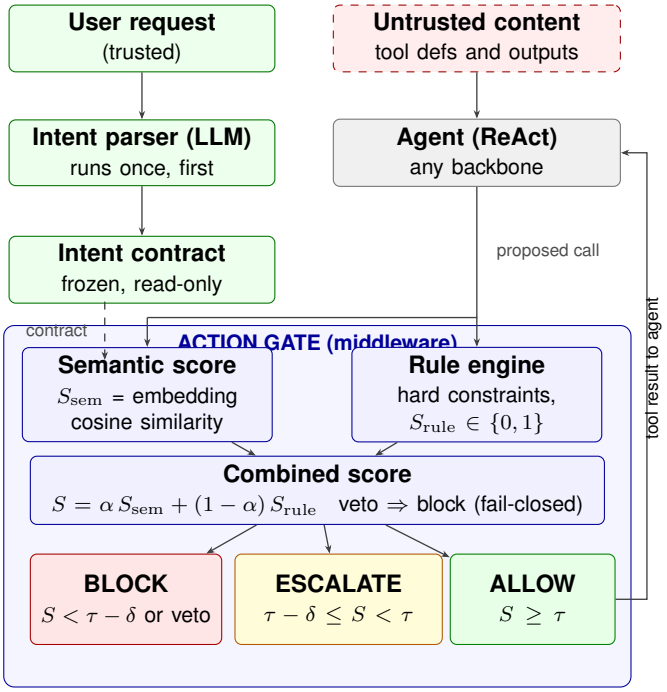


Fig. 1. IntentGate middleware architecture. The intent parser derives a contract once from the trusted request, before any attacker content (tool definitions, tool outputs) is loaded. The gate scores each proposed call with embedding similarity and a hard-rule veto, then decides allow, block, or escalate; per-action enforcement uses no LLM call.

IV. TOOLGATE B2 BASELINE

The primary baseline is a minimal reimplementation of ToolGate [13]: Hoare-style contracts $\{P\}T\{Q\}$ with preconditions P over a symbolic world state, postconditions Q, and snapshot/rollback on post-condition violation. Contracts cover the evaluated tool subset only: InjecAgent 79/79 user and attacker tools (100%), MCPTox 65/801 registered tools (8.1% distinct and 37.2% availability-weighted). Missing contracts are never silently allowed: they are counted as `no_contract` and reported. Fidelity was validated by contract review and recorded-call replay rather than a ToolBench run.

Setup cost is measured directly: ToolGate requires 145

hand-authored contracts on InjecAgent against zero setup for IntentGate, whose contract is derived from the request. On MCPTox, extending contracts toward 84.9% weighted coverage does not meaningfully reduce attack success: the coverage-vs-ASR curve is flat because attack calls look like legitimate queries; the attack is in the poisoned registration, not in calling the wrong tool. ToolGate is statistically indistinguishable from the unprotected baseline on MCPTox, which validates the head-to-head comparison.

V. Evaluation

A. Benchmarks and cases

InjecAgent [25]: indirect prompt injection in tool-integrated agents, 2,108 cases (dh/ds $\times$ base/enhanced), frozen at pinned commits. **MCPTox** [4]: tool poisoning on real-world MCP servers, 1,348 static-snapshot cases. MCPTox results are explicitly scoped to the static snapshot (live-server evaluation is deferred; the poisoning-at-registration logic is identical in both).

B. Metrics

Attack success rate (ASR): share of attack cases where the attacker's objective is achieved. For MCPTox, the same quantity is reported as attack-influenced calls because the heuristic evaluator flags payload or sensitive-path evidence rather than a strict objective. Block rate: share of gated calls blocked or escalated, reported by stratum where stratified. Valid rate: share of legitimate task steps completed, the utility metric. Latency: p50/p95 per-call gate overhead. Setup cost: hand-authored contracts required.

C. Baselines and ablations

B1 is the unprotected agent. B2 is the ToolGate reimplementation. Ours is IntentGate at $\tau = 0.75$, $\delta = 0.1$, $\alpha = 0.7$. The full matrix adds three ablations: A1 semantic-only (no rule veto), A2 rule-only (no embeddings), A3 raw-request embedding (similarity to the request text instead of the parsed contract). All conditions run with identical prompts, tool blocks, agent model (gpt-4o-mini), embedding model (all-MiniLM-L6-v2 [29]), and seed 42.

D. Statistics

All comparisons use the paired McNemar test with continuity correction [30] and seeded bootstrap 95% CIs (10k resamples) per benchmark and per-risk breakdown, with Bonferroni correction across the 10-risk breakdowns ($\alpha = 0.005$). Post-hoc power is 1.0 at $\alpha = 0.005$ for both benchmarks. Results from the frozen matrix are reported as-is; pilot results are labeled preliminary.

VI. Results

A. Main results

Table II summarizes the headline result. IntentGate reduces ASR from 19.0% to 0.7% on InjecAgent and attack-influenced calls from 20.7% to 10.7% on MCPTox (McNemar $p = 1.3\text{e}{-33}$ and $p = 1.25\text{e}{-07}$). The ToolGate baseline reduces ASR only slightly (17.2% vs. 19.0% on InjecAgent) and is not statistically distinguishable from the unprotected agent on MCPTox. Call-level utility is reported with the benign-task pilot in Section VI-F.

TABLE II
Headline results (frozen matrix). MCPTox reports attack-influenced ASR; setup cost counts hand-authored contracts.

Condition	InjecAgent ASR	MCPTox ASR	Setup cost	p95 latency
B1 (none)	19.0%	20.7%	—	—
B2 (ToolGate)	17.2%	20.1%	145 contracts	—
Ours (IntentGate)	**0.7%**	**10.7%**	**0**	16–20 ms

TABLE III
Ablations (frozen matrix). A1–A3 replace gate components; MCPTox reports attack-influenced calls (out of 1,348).

Condition	InjecAgent ASR	MCPTox calls	Notes
A1 (semantic-only)	0.0%	81	Detects more; over-blocks legitimate calls
A2 (rule-only)	14.7%	263	Closest to unprotected on hard slices
A3 (raw-request)	0.7%	116	Equal to ours on InjecAgent; fewer MCPTox calls
Ours (full)	**0.7%**	**144**	Contract adds utility and auditability

B. Setup cost

IntentGate requires zero hand-authored contracts; ToolGate requires 145 on InjecAgent and 65 on MCPTox (frozen matrix), extendable to 583 without closing the gap. This is the metric ToolGate's own paper does not measure.

C. Ablations

Semantic scoring does most of the detecting. The rule-only ablation (A2) is the weakest monitor, with 14.7% ASR on InjecAgent and 263 attack-influenced MCPTox calls, close to the unprotected agent on the hard slices. Semantic-only (A1) detects more than the full gate on MCPTox (81 vs. 144 attack-influenced calls) but over-blocks legitimate calls, as the benign-task pilot shows. The raw-request ablation (A3) matches the full gate on InjecAgent (0.7% both) and detects slightly more MCPTox calls (116 vs. 144). The contract's value is therefore utility and auditability rather than headline ASR. The false-escalate rate is 14.8% on 583 escalated calls.

D. Pareto and threshold

Figure 2 shows the threshold sweep on the pilot data: attack success falls to zero at the operating point $\tau = 0.75$ while the false-positive rate stays at 4%. Cross-split transfer remains future work.

E. Error taxonomy

Thirteen InjecAgent residuals remain: all with $S \in [0.75, 0.79]$, read-only exfil calls just above τ. Intent mimicry

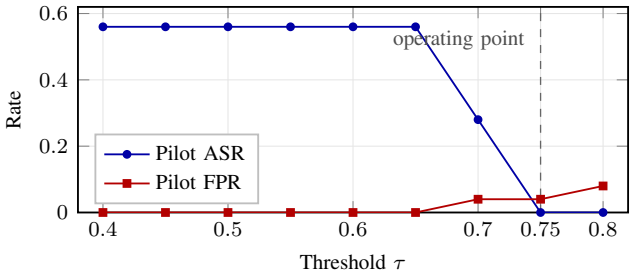


Fig. 2. Threshold sweep on the 200-case pilot ($\delta = 0.1$): attack success rate (ASR) and false-positive rate (FPR) versus τ. The operating point $\tau = 0.75$ sits at the knee: ASR reaches zero while FPR stays at 4%.

TABLE IV
UTILITY AND ROBUSTNESS PILOTS (PRELIMINARY): BENIGN-TASK UTILITY, SECOND BACKBONE, AND PROMPT-LEVEL BASELINES.

Pilot	Result
E23 benign utility	user-tool allowed: none 16.0%, ours 12.0%, semantic-only 8.0%; clean-call block: none 0.0%, ours 25.0%, semantic-only 42.9%
E18 second backbone	qwen3.5:4b: ASR 27.1% $\rightarrow$ 2.1%
E24 prompt baselines	delimiters 4.3%, sandwich 4.3%, no defense 6.8%, instructional prevention 8.9% (50 cases each)

inflates S by only +0.0112 on average, with 0/13 cases newly crossing τ, so the semantic threshold is stable under rephrasing. MCPTox residuals cluster in poisoned-registration classes no contract covers.

F. Validation results (preliminary)

E18 replicates the ASR reduction on a second backbone, qwen3.5:4b (27.1% $\rightarrow$ 2.1%); gpt-6-luna is excluded because its prose-only output never participates in the ReAct-style loop. Block rate by stratum: specific 12.8%, vague 9.6%, overall 12.4%. The benign-task pilot (50 clean cases) compares the full gate with the unprotected agent and the semantic-only ablation: user-tool allowed 16.0%, 12.0%, and 8.0% respectively, and clean-call block 0.0%, 25.0%, and 42.9%. Semantic-only over-blocks legitimate calls, so the contract preserves utility in this comparison. The prompt-baseline pilot (50 attack cases per condition) gives ASR 6.8% with no defense prompt, 4.3% with delimiters, 4.3% with sandwiching, and 8.9% with instructional prevention. A third backbone and adaptive-attacker robustness remain in progress; all pilot numbers are preliminary.

VII. LIMITATIONS

The primary claims are scoped to static attacks; adaptive-attacker robustness is reported as ongoing work rather than claimed. MCPTox results are on a static snapshot, not live servers. The main matrix uses a single backbone (gpt-4o-mini); qwen3.5:4b replicates the delta, and gpt-6-luna is incompatible with the shared ReAct-style prompt. The parser is an LLM call once per session, included in the cost; per-action enforcement uses no LLM. Contract coverage on the MCPTox long tail remains low by design (missing contracts are counted as `no_contract` and never silently allowed). The valid rate measures call-level allowance of the user's tool call, not end-to-end task success; the benign-task pilot is a 50-case comparison and should be read as such. Block rate includes escalate candidates and is not a pure false-positive rate. The MCPTox attack-influenced metric comes from a heuristic evaluator (payload and sensitive-path evidence), which is coarser than a strict objective check.

VIII. CONCLUSION

This paper shows that an action gate can derive its policy automatically from the user's request, enforce it with embedding similarity plus a deterministic veto at the tool-call chokepoint, and operate with zero setup cost, 16–20 ms p95 latency, and a large measured reduction in attack success on both an injection and a tool-poisoning benchmark. The reproducible artifact is released after acceptance. Future work covers adaptive-attacker hardening, live-server MCPTox evaluation, and multi-turn drift.

ACKNOWLEDGMENT

Large language models were used for drafting assistance, experiment code generation, and result summarization in preparing this paper; their use was limited to non-intellectual preparation tasks. All experimental results, statistical claims, and the final manuscript were reviewed and verified by the authors; the LLM was not used to fabricate data.

REFERENCES

[1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *The Eleventh International Conference on Learning Representations (ICLR)*, 2023.

[2] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023, pp. 79–90.

[3] F. Perez and I. Ribeiro, "Ignore previous prompt: Attack techniques for language models," 2022, neurIPS 2022 ML Safety Workshop. [Online]. Available: https://arxiv.org/abs/2211.09527

[4] Z. Wang, Y. Gao, Y. Wang, S. Liu, H. Sun, H. Cheng, G. Shi, H. Du, and X.-Y. Li, "MCPTox: A benchmark for tool poisoning on real-world MCP servers," in *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, 2026, pp. 35 811–35 819. [Online]. Available: https://ojs.aaai.org/index.php/AAAI/article/view/40895

[5] B. Radosevich and J. Halloran, "MCP safety audit: LLMs with the Model Context Protocol allow major security exploits," 2025, preprint. [Online]. Available: https://arxiv.org/abs/2504.03767

[6] OWASP Foundation, "OWASP top 10 for large language model applications," 2025, available: https://genai.owasp.org/llm-top-10/.

[7] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, and F. Tramèr, "Defeating prompt injections by design (CaMeL)," 2025, preprint. [Online]. Available: https://arxiv.org/abs/2503.18813

[8] S. Chen, J. Piet, C. Sitawarin, and D. Wagner, "StruQ: Defending against prompt injection with structured queries," in *34th USENIX Security Symposium (USENIX Security)*, 2025, pp. 2383–2400. [Online]. Available: https://www.usenix.org/conference/usenixsecurity25

[9] S. Chen, A. Zharmagambetov, S. Mahloujifar, K. Chaudhuri, D. Wagner, and C. Guo, "SecAlign: Defending against prompt injection with preference optimization," in *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2025, pp. 2833–2847.

[10] E. Wallace, K. Xiao, R. Leike, L. Weng, J. Heidecke, and A. Beutel, "The instruction hierarchy: Training LLMs to prioritize privileged instructions," 2024, preprint. [Online]. Available: https://arxiv.org/abs/2404.13208

[11] K. Hines, G. Lopez, M. Hall, F. Zarfati, Y. Zunger, and E. Kiciman, "Defending against indirect prompt injection attacks with spotlighting," 2024, preprint. [Online]. Available: https://arxiv.org/abs/2403.14720

[12] T. Rebedea, R. Dinu, M. Sreedhar, C. Parisien, and J. Cohen, "NeMo guardrails: A toolkit for controllable and safe LLM applications with programmable rails," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations (EMNLP)*, 2023, pp. 431–445.

[13] Y. Liu, X. Peng, J. Cao, X. Wang, S. Deng, J. Chen, J. Yin, and X. Zhang, "ToolGate: Contract-grounded and verified tool execution for LLMs," 2026, preprint. [Online]. Available: https://arxiv.org/abs/2601.04688

[14] T. Shi, J. He, Z. Wang, H. Li, L. Wu, W. Guo, and D. Song, "Progent: Securing AI agents with privilege control," 2025, preprint. [Online]. Available: https://arxiv.org/abs/2504.11703

[15] W. Zhao, Z. Li, P. Zhang, and J. Sun, "ClawGuard: A runtime security framework for tool-augmented LLM agents against indirect prompt injection," 2026, preprint. [Online]. Available: https://arxiv.org/abs/2604.11790

[16] F. Jia, T. Wu, X. Qin, and A. Squicciarini, "The Task Shield: Enforcing task alignment to defend against indirect prompt injection in LLM agents," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL), Volume 1: Long Papers*, 2025, pp. 29 680–29 697.

[17] A. Zou, Z. Wang, N. Carlini, M. Nasr, J. Z. Kolter, and M. Fredrikson, "Universal and transferable adversarial attacks on aligned language models," 2023, preprint. [Online]. Available: https://arxiv.org/abs/2307.15043

[18] Y. Ruan, H. Dong, A. Wang, S. Pitis, Y. Zhou, J. Ba, Y. Dubois, C. J. Maddison, and T. Hashimoto, "Identifying the risks of LM agents with an LM-emulated sandbox," in *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.

[19] J. Yi, Y. Xie, B. Zhu, E. Kiciman, G. Sun, X. Xie, and F. Wu, "Benchmarking and defending against indirect prompt injection attacks on large language models," in *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, 2025, pp. 1809–1820.

[20] K. Zhu, X. Yang, J. Wang, W. Guo, and W. Y. Wang, "MELON: Provable defense against indirect prompt injection attacks in AI agents," in *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025. [Online]. Available: https://arxiv.org/abs/2502.05174

[21] J. Kim, W. Choi, and B. Lee, "Prompt flow integrity to prevent privilege escalation in LLM agents," 2025, preprint. [Online]. Available: https://arxiv.org/abs/2503.15547

[22] W. Hua, X. Yang, M. Jin, Z. Li, W. Cheng, R. Tang, and Y. Zhang, "TrustAgent: Towards safe and trustworthy LLM-based agents," in *Findings of the Association for Computational Linguistics: EMNLP 2024*, 2024. [Online]. Available: https://aclanthology.org/2024.findings-emnlp.585/

[23] L. Beurer-Kellner, B. Buesser, A.-M. Creţu, E. Debenedetti, D. Dobos, D. Fabian, M. Fischer, D. Froelicher, K. Grosse, D. Naeff, E. Ozoani, A. Paverd, F. Tramèr, and V. Volhejn, "Design patterns for securing LLM agents against prompt injections," 2025, preprint. [Online]. Available: https://arxiv.org/abs/2506.08837

[24] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024. [Online]. Available: https://arxiv.org/abs/2406.13352

[25] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, "InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents," in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024. [Online]. Available: https://aclanthology.org/2024.findings-acl.624/

[26] H. Zhang, J. Huang, K. Mei, Y. Yao, Z. Wang, Q. Zhan, H. Wang, and Y. Zhang, "Agent Security Bench (ASB): Formalizing and benchmarking attacks and defenses in LLM-based agents," in *International Conference on Learning Representations (ICLR)*, 2025. [Online]. Available: https://arxiv.org/abs/2410.02644

[27] Anthropic, "Introducing the Model Context Protocol," 2024, available: https://www.anthropic.com/news/model-context-protocol.

[28] Q. Zhan, R. Fang, H. S. Panchal, and D. Kang, "Adaptive attacks break defenses against indirect prompt injection attacks on LLM agents," in *Findings of the Association for Computational Linguistics: NAACL 2025*, 2025, pp. 7101–7117. [Online]. Available: https://aclanthology.org/2025.findings-naacl.395/

[29] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using siamese BERT-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 2019, pp. 3980–3990.

[30] T. G. Dietterich, "Approximate statistical tests for comparing supervised classification learning algorithms," *Neural Computation*, vol. 10, no. 7, pp. 1895–1923, 1998.